

PYTHON-BASED WEB VULNERABILITY SCANNER

Project Report

Technology: Python

Project Type: Cybersecurity / Defensive Security

Testing Scope: Authorized and controlled web applications

Prepared as a Student Cybersecurity Project

1. Abstract

The Python-Based Web Vulnerability Scanner is a lightweight defensive security tool designed to identify selected common security weaknesses in authorized web applications. The application sends controlled HTTP requests and analyzes response headers, cookies, server information, and HTML forms. Detected findings are categorized by severity and accompanied by remediation recommendations. The scanner produces a structured JSON report, making the results easier to review and document.

2. Introduction

Modern web applications depend on secure server configuration, appropriate HTTP security controls, and safe handling of sensitive information. Configuration mistakes can expose applications to security risks. Automated vulnerability scanning can help developers and security students identify such weaknesses early.

This project implements a small modular scanner in Python. It focuses on defensive assessment rather than exploitation and is intended for applications owned by the tester or for which explicit testing permission has been granted.

3. Problem Statement

Manual checking of web security configurations can be repetitive and error-prone. A lightweight automated tool is therefore useful for inspecting selected HTTP and HTML security properties and generating consistent findings.

4. Objectives

- Develop a web vulnerability scanner using Python.
- Inspect important HTTP security headers.
- Identify insecure cookie attributes.
- Detect basic information disclosure through response headers.
- Analyze HTML forms for selected insecure configurations.
- Categorize findings by severity.
- Generate structured vulnerability reports.
- Provide remediation recommendations.

5. Scope of the Project

The scanner covers a limited set of defensive checks: security headers, cookie security attributes, server and technology disclosure, and a basic password-form method check. It is not intended to replace professional penetration-testing tools or a full security audit.

6. Technologies Used

Technology	Purpose
Python 3	Core programming language
Requests	HTTP requests and response analysis

BeautifulSoup	HTML and form parsing
argparse	Command-line interface
JSON	Structured vulnerability reporting

7. System Architecture

The application follows a modular architecture. The main scanner obtains the HTTP response and passes it to independent analysis modules.

Target Application → HTTP Request → Security Analysis Modules → Finding Engine → JSON Report

- HTTP Scanner: connects to the authorized target and obtains the response.
- Header Analyzer: checks selected security-related HTTP headers.
- Cookie Analyzer: examines Secure and HttpOnly attributes.
- Information Analyzer: identifies potentially unnecessary server or technology disclosure.
- Form Analyzer: parses HTML forms and checks selected sensitive-form configurations.
- Report Generator: stores findings with category, severity, description, and recommendation.

8. Project Modules

HTTP Scanner — Handles URL requests, status codes, redirects, timeouts, and connection errors.

Security Header Analyzer — Checks Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Referrer-Policy, and Permissions-Policy.

Cookie Analyzer — Checks whether cookies use Secure and HttpOnly attributes.

Information Disclosure Analyzer — Checks Server and X-Powered-By response headers.

HTML Form Analyzer — Identifies password-containing forms that use GET.

Report Generator — Creates a JSON report containing scan details and findings.

9. Methodology

- Validate and receive the target URL from the command line.
- Send a controlled HTTP GET request with a scanner identification User-Agent.
- Record the HTTP status code and final URL.
- Run each security analysis module against the response.
- Store detected findings in a common data structure.
- Display findings in the terminal.
- Export the final results to reports/report.json.

10. Vulnerability Checks

Check	Severity Used	Purpose
Missing CSP	Medium	Identifies absence of a Content Security Policy.
Missing X-Frame-Options	Medium	Identifies missing clickjacking-related response protection.
Missing X-Content-Type-Options	Medium	Identifies missing MIME-sniffing protection.
Missing Referrer-Policy	Medium	Identifies absence of explicit referrer handling.
Missing Permissions-Policy	Medium	Identifies absence of explicit browser feature policy.
Cookie without Secure	Medium	Flags cookies that are not restricted to secure transport.
Cookie without HttpOnly	Medium	Flags cookies accessible to client-side scripts.
Server information exposed	Low	Identifies potentially unnecessary server disclosure.
X-Powered-By exposed	Low	Identifies technology disclosure.
Password form using GET	Medium	Flags sensitive form data that may appear in URLs.

11. Testing Plan

ID	Test Condition	Expected Result
TC01	All selected headers present	No corresponding missing-header finding
TC02	CSP absent	Medium finding
TC03	X-Frame-Options absent	Medium finding
TC04	Server header exposed	Low finding
TC05	Cookie lacks Secure	Medium finding
TC06	Password form uses GET	Medium finding
TC07	Invalid/unreachable URL	Connection error handled
TC08	Hardened test application	Fewer findings

12. Expected Output

The terminal displays the target, HTTP status, detected findings, severity, descriptions, and recommendations. A JSON file is created in the reports directory for further review or integration with future reporting features.

13. Advantages

- Simple and modular Python implementation.
- Easy to extend with additional security checks.
- Produces structured machine-readable output.
- Useful for learning HTTP and web security concepts.
- Suitable for controlled cybersecurity labs and authorized assessments.

14. Limitations

- The scanner performs only a limited number of checks.
- It does not provide a complete vulnerability assessment.
- It does not exploit vulnerabilities.
- Some findings may be false positives or require manual verification.
- A successful scan does not prove that an application is secure.
- Testing must be restricted to systems for which authorization exists.

15. Future Enhancements

- Add HTTPS/TLS configuration analysis.
- Add additional OWASP-aligned checks in a controlled lab.
- Create an HTML report with charts and severity summaries.
- Add SQLite storage for scan history.
- Build a Flask or Streamlit dashboard.
- Add configurable scan profiles.
- Add unit and integration tests.
- Add authenticated scanning for applications where permission is explicitly provided.

16. Conclusion

The Python-Based Web Vulnerability Scanner demonstrates how Python can be applied to practical defensive cybersecurity tasks. The project combines HTTP communication, response analysis, HTML parsing, security configuration checks, modular programming, and automated reporting. Although it is intentionally limited in scope, it provides a strong foundation for developing a more comprehensive security assessment platform.

17. Ethical and Legal Considerations

The scanner should only be used against systems owned by the student or systems for which explicit authorization has been granted. Unauthorized vulnerability scanning can disrupt services and may violate laws, contracts, or organizational policies. For demonstrations and academic work, local applications and intentionally vulnerable training environments should be preferred.

18. Suggested Project Demonstration

- Run the scanner against a local test application.
- Show the terminal scan results.
- Open reports/report.json.
- Demonstrate one intentionally misconfigured security header.
- Fix the configuration and run the scanner again.

- Compare the before-and-after results.
- Explain the limitations and future improvements.